

Test Design Report: CodePop

Version: Final (Spring 2026)

Team: SocialDrinkers (3)

- [Test Design Report: CodePop](#)
 - [1. Introduction: Testing Journey & Mindset](#)
 - [2. Testing Approach: How We Tested](#)
 - [2.1 Backend Isolation Testing](#)
 - [2.2 Frontend API & Interaction Testing](#)
 - [3. System-Level \(End-to-End\) Testing](#)
 - [3.1 Automated Multi-Instance Orchestration](#)
 - [3.2 Reproducing Results](#)
 - [3.3 Manual User Acceptance Testing \(UAT\)](#)
 - [4. What We Learned and Fixed](#)
 - [4.1 The Importance of Automated Integration Testing](#)
 - [4.2 Vulnerability Discovery: RBAC Security Gap](#)
 - [4.3 Resolution: Serializer Redaction and Permission Enforcement](#)
 - [4.4 Discovery: Server-to-Server Authentication Bottlenecks](#)
 - [4.5 Challenge: Leader Election and Network Convergence](#)
 - [5. Challenges & Areas of Concern](#)
 - [5.1 Distributed System Testing](#)
 - [5.2 Frontend UI Automation](#)
 - [5.3 Reflections on RBAC Design](#)
 - [5.4 Current Worries](#)
 - [6. Code Coverage & Metrics](#)
 - [6.1 Backend Coverage](#)
 - [6.2 Frontend Coverage](#)
 - [7. Final Results & Presentation Readiness](#)
 - [7.1 Final Execution Results](#)
 - [7.2 Instructions for the Presentation Team](#)
 - [8. Appendices \(Screenshots & Logs\)](#)
 - [8.1 Automated Multi-Instance Orchestration Log](#)
-

1. Introduction: Testing Journey & Mindset

Our testing journey for CodePop has been a transformative experience for our team. We moved from viewing testing as a checkbox requirement to seeing it as the fundamental bedrock of our development lifecycle.

Early on, we deeply learned about the importance of automated tests when developing a project of this scale. As our system evolved into a complex, decentralized Peer-to-Peer (P2P) network, our mindset shifted from local verification to system-wide resilience. We recognized that in a distributed environment, "success"

is defined not just by individual components working correctly, but by the network's ability to maintain consistency, security, and availability under real-world conditions.

This report details how our automated suites caught critical security vulnerabilities and guided our architectural decisions, ensuring that CodePop is robust and ready for its final delivery.

2. Testing Approach: How We Tested

To ensure stability across our decentralized architecture, we adopted a tiered testing strategy:

- **Isolation Layer (Backend Unit):** We used Django `unittest` and `APITestCase` to validate core models (Inventory, Orders, Users) and security logic in a controlled environment.
- **Interaction Layer (Frontend API):** Utilizing a custom TypeScript test runner (`api_tests.ts`), we verified that the frontend models correctly interface with the backend API and handle real-world data shapes.
- **Orchestration Layer (System Integration):** We employed a custom multi-instance framework to simulate complex P2P conditions, including cross-server authentication and regional data aggregation.

2.1 Backend Isolation Testing

- **Framework:** Django `unittest` and `REST Framework APITestCase`.
- **Strategy:** We used `unittest.mock` to simulate network traffic for fast logic verification, allowing for rapid iteration without needing a live network for every test.
- **Scope:**
 - **Model & Logic:** Validated core data models (Inventory, Orders, Users) and business logic.
 - **P2P Security:** Verified RS256 asymmetric signature validation and public key exchange logic in isolation.
 - **Authentication:** Tested JWT claim verification and proxy authentication logic.
- **Automation:** Integrated into the development workflow (`python manage.py test`) to catch regressions during daily coding.

2.2 Frontend API & Interaction Testing

- **Framework:** Custom TypeScript Test Runner (`api_tests.ts`).
- **Strategy:** Conducted end-to-end API verification from the frontend's perspective.
- **Scope:**
 - Authentication flows (Login, Register, Profile Management).
 - Business logic (Drink creation, Ordering, Inventory management).
 - Cross-cutting features (Chatbot, Notifications, Revenue).
 - Admin/P2P controls (Server discovery, Master list access).
- **Automation:** Executed via `npm run ts-node models/tests/api_tests.ts` to ensure frontend-to-backend compatibility.

3. System-Level (End-to-End) Testing

While unit testing accounts for how the system works in isolation, it cannot simulate the complexities of our distributed architecture. Our system-level testing orchestrates live server instances, real database interactions, and authentic HTTP communication.

3.1 Automated Multi-Instance Orchestration

Our core integration verification is handled by `full_p2p_automated_test.py`. This script automates a high-fidelity, system-wide environment:

- **Dynamic Infrastructure:** Spins up three independent Django servers (A, B, and C) on dynamic ports and three separate PostgreSQL databases to ensure zero state-leakage.
- **Auto-Discovery & Sync:** Verifies that servers can cross-register using RSA public keys and that the global `MasterList` correctly propagates.
- **Cross-Server Flows:** Tests "Roaming" user scenarios where a user registered on Server A logs in and performs actions on Server B, which are then proxied back to their home server.
- **Full-Stack Features:** Verifies Stripe integration, Machine proxying, Global aggregation, and Fault tolerance (terminating a node to verify network resilience).

3.2 Reproducing Results

To reproduce our system-level test results, follow these steps:

1. **Activate Environment:** `source codepop_virtual_enviroment/bin/activate`
2. **Run Orchestration Suite:** `python codepop_backend/integration_tests/full_p2p_automated_test.py`
3. **Verify Output:** The script will execute 17 sections of tests. A final summary log will appear at the end.
4. **Expected Outcome:** All 17 verification steps (Auth, Proxying, Stripe, Aggregation, etc.) must return "SUCCESS".

3.3 Manual User Acceptance Testing (UAT)

While our automated suites provide a rigorous baseline, we perform manual User Acceptance Testing to ensure the system meets our high standards for user experience and visual feedback. These scenarios simulate real-world usage patterns across the decentralized network.

Scenario	Prerequisites	Execution Steps	Expected Result
1. Roaming Authentication & Profile Persistence	A user account registered on Server A.	1. Access the login page of Server B. 2. Authenticate using Server A credentials. 3. Navigate to the "Profile" page.	Access is granted via P2P proxy; the user's profile data and preferences from Server A are displayed accurately on Server B.
2. Cross-Server Distributed Soda Catalog	Featured sodas exist on Server A but not on Server B.	1. Log in to Server B as a Server A user. 2. Open the drink creation menu.	The menu dynamically lists sodas and ingredients from the user's Home Server (A) through the distributed proxy layer.
3. End-to-End Transactional Integrity	Stripe Mocking is enabled; Inventory exists.	1. Build a custom drink. 2. Proceed to checkout.	The order is processed; Inventory counts are decremented locally; Revenue

Scenario	Prerequisites	Execution Steps	Expected Result
		3. Enter mock payment details.	records are generated; Order appears in History.
4. Regional Logistics Oversight	Multiple servers active in a single region.	1. Log in as a Logistics Manager. 2. Access the "Regional Inventory" dashboard.	The manager can view aggregated inventory levels from all stores in their region and trigger manual stock updates via proxy.
5. Super Admin Global Governance	Multiple servers active across multiple regions.	1. Log in as Super Admin. 2. Access the "Global Revenue" and "Server Registry" views.	Aggregated revenue data from all global regions is retrieved; The status and health of all peer nodes are visible in real-time.

4. What We Learned and Fixed

The testing sprint provided invaluable insights into our system's behavior and helped us identify and resolve critical issues before the final presentation.

4.1 The Importance of Automated Integration Testing

The introduction of our **Integration Tests Suite** was a turning point. We have been updating it continuously to ensure that nearly every feature in the project is tested in a live environment. This suite was not just a tool for verification; it became our primary discovery mechanism for architectural flaws and security gaps.

4.2 Vulnerability Discovery: RBAC Security Gap

While running our integration tests earlier this week, we uncovered a significant security vulnerability: the **inventory report API had its RBAC (Role-Based Access Control) security configuration mistakenly removed**.

Because of this oversight, Customers or Guests could have accessed full, detailed inventory reports just by knowing the endpoint URL. This kind of vulnerability, while seemingly minor in a local environment, would have been a major risk in a production-ready decentralized system.

4.3 Resolution: Serializer Redaction and Permission Enforcement

Upon discovery, we were able to quickly fix this on the backend with targeted edits:

- Permission Enforcement:** Re-applied `permission_classes` to the `InventoryReportAPIView` to restrict access to authorized roles (Managers and Peer Servers).
- Serializer Redaction:** Updated the `InventorySerializer` to proactively redact sensitive information. If a Customer or Guest attempts to access inventory data, the serializer now removes fields like `Quantity` and `ThresholdLevel`, returning only an `InStock` boolean status.

4.4 Discovery: Server-to-Server Authentication Bottlenecks

Our integration tests also revealed a bottleneck in how **Global Aggregation** worked. Initially, the Super Admin's JWT was being proxied to all peer servers during revenue aggregation. However, we found that this created a "trust chain" issue: a peer server in one region might not immediately have the public key required to verify a JWT issued by a server in a different region.

The Fix: We pivoted to a dedicated **Server-Token** authentication scheme for inter-server calls. By introducing a **RevenuesPeerView** that uses **IsPeerServer** permissions, we decoupled administrative user sessions from the low-level data synchronization, making the global dashboard significantly more reliable and faster to load.

4.5 Challenge: Leader Election and Network Convergence

Testing the P2P synchronization logic exposed a "split-brain" risk where multiple servers in the same region could both claim to be the "Leader" if they were started simultaneously. This caused data duplication during the master-list sync.

What we learned: We refined our server setup logic to explicitly distinguish between "Creating" a region and "Joining" one. A joining server is now strictly forbidden from becoming a leader during its initial handshake, ensuring that the network always converges on a single source of truth for regional data. This fix was verified by our multi-instance test script, which ensures that complex P2P scenarios—like roaming and aggregation—function correctly under this refined setup logic.

5. Challenges & Areas of Concern

5.1 Distributed System Testing

The most significant challenge was verifying roaming user functionality. Maintaining location-agnostic access required a complex proxying architecture rather than simple database replication. Simulating real-time network requests between instances to ensure data consistency was a hurdle that only our multi-instance orchestration suite could reliably clear.

5.2 Frontend UI Automation

A major technical challenge we faced was the portability of frontend UI tests. While we successfully developed a large-scale test suite using Jest and Playwright, we encountered significant environment-dependency issues when attempting to run it across different developer machines. The complexity of configuring headless browsers and consistent timing across different hardware meant that we could not reliably integrate these tests into our shared CI/CD workflow within the project's timeline. This remains a key area for future infrastructure improvement.

5.3 Reflections on RBAC Design

The discovery of the RBAC vulnerability led us to reflect on our overall security architecture. If we were to start this project over, we would change several things about our RBAC implementation. Specifically, we would prefer making the **proxying part of it a middleware**, in a similar fashion to how the authentication permissions middleware is handling it. This would consolidate the logic and reduce the risk of accidentally omitting security checks on individual views.

5.4 Current Worries

Despite our rigorous testing, some areas still warrant caution:

- **Network Latency during P2P Discovery:** High latency environments might still expose timing issues in initial server handshakes.
- **Race Conditions in High-Volume Inventory Updates:** Ensuring total consistency during simultaneous heavy ordering remains a point of concern.
- **AI Logic Edge Cases:** Validating that the AI system handles complex, overlapping requests without error.

6. Code Coverage & Metrics

Layer	Unit Tests	Integration Tests	Overall Coverage (Combined)
Backend	~70%	~50% (Overall) / 95%+ (P2P)	~72%
Frontend	0%	~20% (API/Model Layer)	~20%

6.1 Backend Coverage

We maintained approximately 72% overall code coverage for the backend. We prioritized the P2P networking and security logic, which has **95%+ coverage** via the orchestration suite. Standard Django unit tests handle model validation and CRUD operations.

6.2 Frontend Coverage

Frontend coverage is at **~20%**, focused on the API and Model layer. We rely on our TypeScript test runner to exercise frontend communication.

It is important to note that a significant effort was made to implement a comprehensive frontend automated testing suite using **Jest and Playwright**. This suite was designed to provide deep coverage of the UI and user interactions. However, due to time constraints and environment-specific dependencies that limited its execution to a single development machine, we were unable to merge this suite into the master branch for the final release. Consequently, the React UI presentation layer is primarily verified through manual testing and User Acceptance Tests (UAT), while the automated metrics reflect only the API/Model layer.

7. Final Results & Presentation Readiness

This section summarizes our final testing results and provides instructions for the team handling the final presentation to ensure they can successfully reproduce our findings and demonstrate the application's functionality.

7.1 Final Execution Results

In our final pre-presentation run, the automated orchestration suite passed all 17 sections successfully. Our regression tests for both the backend and frontend were completed with no new failures, confirming that our recent security fixes did not introduce regressions in core business logic.

7.2 Instructions for the Presentation Team

To ensure a smooth demonstration, the following steps should be followed to reproduce the system-level results and verify the core P2P features of CodePop:

1. Run the Automated Orchestration Suite

- **Action:** Execute `python codepop_backend/integration_tests/full_p2p_automated_test.py` from the project root.
- **Expected Outcome:** All 17 sections pass, showing a clean, functional network across three nodes.

2. Demonstrate Roaming Authentication

- **Action:** Authenticate on a visiting server (e.g., Server B) using credentials for a user whose account belongs to a different home server (e.g., Server A).
- **Expected Outcome:** The proxy login succeeds, and the user's role-based access is correctly preserved.

3. Verify Cross-Server Business Flows

- **Action:** Perform profile updates or place an order while connected to a visiting server.
- **Expected Outcome:** The operations are correctly proxied and synchronized back to the home server.

4. Showcase RBAC Security (The "Fixed" Vulnerability)

- **Action:** Attempt to access the `/backend/inventory/report/` endpoint as a Customer.
- **Expected Outcome:** The system returns a `403 Forbidden` or provides redacted data (only showing `InStock` status), proving that sensitive counts are no longer exposed.

5. Demonstrate Global Aggregation

- **Action:** Log in as a Super Admin and access the Global Revenue dashboard.
- **Expected Outcome:** The dashboard successfully pulls and aggregates data from all regional nodes via the `Server-Token` authentication scheme.

8. Appendices (Screenshots & Logs)

8.1 Automated Multi-Instance Orchestration Log

```
===== Initializing Port Discovery =====
Using dynamic ports: Server A = 8050, Server B = 8051, Machine = 8052, Server C = 8053

===== Starting FULL AUTOMATED P2P INTEGRATION TEST (AUTO-DISCOVERY) =====
Generating unique RSA keypair for Server A...
Generating unique RSA keypair for Server B...
Generating unique RSA keypair for Server C...
Generating unique RSA keypair for Server Malicious...

[1/16] Setting up databases and self-identification...
Ensuring PostgreSQL database 'p2p_test_a' exists...
Ensuring PostgreSQL database 'p2p_test_b' exists...
Ensuring PostgreSQL database 'p2p_test_c' exists...
Database 'p2p_test_b' not found. Creating...
Database 'p2p_test_c' not found. Creating...
Database 'p2p_test_a' not found. Creating...
Database 'p2p_test_b' created.
Database 'p2p_test_c' created.
Database 'p2p_test_a' created.

[2/16] Launching background servers...
Spawning Server A on port 8050...
Spawning Server B on port 8051...
Spawning Server C on port 8053...
Spawning Pseudo Machine Server on port 8052 (Test Mode: True)...
Waiting for servers to be ready (timeout 20s)...
Server A is READY.
Server B is READY.
Server C is READY.

[3/16] Performing Cross-Registration via AUTO-DISCOVERY...

[4/16] Creating users across all servers and syncing...
Triggering MasterList sync from Server A...
Verifying MasterList sync on Server C...
SUCCESS: MasterList correctly synchronized (proven by successful roaming logins on Server C).

[5/16] TESTING AUTHENTICATION FLOWS...
Test A: Correct Password Proxy Login (B -> A)...
SUCCESS: Proxy logins worked and returned authoritative tokens from Server A via cookies.
Test A.1: Using Proxied Token on Visiting Server (B)...
SUCCESS: Server B correctly verified and accepted Server A's proxied token.
Test A.2: Role-Based Access Control (RBAC) on Visiting Server (B)...
SUCCESS: Server B correctly granted 'super_admin' access and denied 'customer' access based on P2P tokens.
Test B: Incorrect Password Proxy Login (B -> A)...
SUCCESS: Server correctly rejected wrong password.

[6/16] TESTING CROSS-SERVER JWT VERIFICATION...
Fetching valid token from Server A...
Test C: Using Server A's Token on Server B (Cross-Verification)...
SUCCESS: Server B verified Server A's token using Server A's Public Key.
Verified Identity: traveler_joe
Test D: Using Maliciously Signed Token on Server B...
SUCCESS: Server B correctly blocked malicious token (403).
Test D.1: Using an Expired Token on Server B...
SUCCESS: Server B correctly rejected the expired token (403).

[7/16] TESTING TOKEN REFRESH AND LOGOUT PROXYING
Test E: Token Refresh on Home Server (A)...
SUCCESS: Home server correctly refreshed token via HttpOnly cookie.
Test F: Token Refresh Proxy (B -> A)...
SUCCESS: Server B proxied the refresh request to Server A using cookies.
SUCCESS: Proxied access token (via cookie refresh) is valid on Server B.
Test G: Logout Proxy (B -> A)...
SUCCESS: Server B proxied the logout/blacklist request to Server A and cleared local cookie.
SUCCESS: Logout proxying and cookie clearing verified.

[8/16] TESTING CROSS-SERVER REGISTRATION
Test H: Registering a brand new user directly on Server B...
SUCCESS: Server B registered the new user and issued a token claiming ownership via cookies.

[9/16] TESTING ORDER CREATION AND STRIPE WEBHOOKS
Test I: End-to-End Order Creation and Payment Webhook...
SUCCESS: Payment success and Revenue creation verified.
Simulating Refund Webhook...
SUCCESS: Refund webhook and state transition verified.
SUCCESS: Full order creation, payment intent, and webhook flow passed.

[10/16] TESTING MACHINE STATUS PROXY
Test J: Fetch Machine Status from Proxy Endpoint...
SUCCESS: Machine proxy successfully fetched data from pseudo server.
Test J.1: Test Machine Offline Handling...
SUCCESS: Machine proxy gracefully handled offline machine (503).
Test J.2: Verify Machine 'Run Test' (Fix) Flow...
Spawning Pseudo Machine Server on port 8052 (Test Mode: True)...
SUCCESS: Machine 'run-test' successfully cleared faults.

[11/16] Testing Data Proxying and Syncing (Roaming User)...
SUCCESS: Roaming user authenticated on Visiting Server B.
Testing Profile Patch Proxying (B -> A)...
SUCCESS: Profile update proxied correctly to Home Server.
Testing Preference Creation Proxying (B -> A)...
SUCCESS: Preference creation proxied correctly to Home Server.
Testing Order Creation Syncing (B -> A) with automatic Drink sync...
SUCCESS: Order and related Drinks successfully synced back to Home Server.
Testing Order History Merge (B reads local + remote A)...
SUCCESS: Order history merge successful. Found 3 orders.

[12/16] Populating extra Order and Revenue data for aggregate testing (via API)...
SUCCESS: Extensive Order & Revenue data populated via API across all servers.

[13/16] Testing Aggregate Views...
SUCCESS: Global Revenue Aggregate fetched successfully. Servers included: 3
SUCCESS: Data completeness verified (includes Server A, B, C)

[14/16] Testing Cross-Server Request Proxying...
SUCCESS: Cross-Region read (aggregate inventory) worked for Logistics Manager.
SUCCESS: Home Server routing via proxy worked for Store Manager.

[15/16] Comprehensive RBAC Enforcement Tests...
SUCCESS: RBAC: Store Manager Isolation enforced (403 on aggregate).
SUCCESS: RBAC: Customer Restrictions enforced (403 on inventory report).
SUCCESS: RBAC: Admin Privilege Boundaries enforced (403 on global aggregate).
SUCCESS: RBAC: Repair Staff Access verified (200 on machines, 403 on revenue).

[16/16] Additional Resilience Tests (Fault Tolerance)...
Temporarily terminating Server C...
SUCCESS: System gracefully handled offline node with status 200.

===== Cleaning up processes and databases =====
Terminated background server process.
Terminated background server process.
Terminated background server process.
Dropping PostgreSQL database 'p2p_test_a'...
Database 'p2p_test_a' dropped.
Dropping PostgreSQL database 'p2p_test_b'...
Database 'p2p_test_b' dropped.
Dropping PostgreSQL database 'p2p_test_c'...
Database 'p2p_test_c' dropped.
Done.

-----
Ran 22 test scenarios in 44.198s

SUCCESS: VERIFICATION COMPLETE: All P2P Security and Auth Scenarios Passed.
```